# UMBC Parking Management System
# Database Design Proposal

Mateo Jacome, Jason Chen

Mar 3 2026

# HW #1 Executive Summary

## 1.1 Problem Statement

Parking at a large commuter campus such as UMBC is inefficient, stressful, and difficult to enforce. Students frequently waste valuable time searching for open spaces. Visitors struggle to locate valid parking without clear guidance. Faculty and staff require access to restricted or premium areas. Meanwhile, enforcement is largely reactive and labor-intensive, relying on officers instead of automated validation mechanisms.

The core problem is the absence of a centralized, real-time database system that integrates parking permits, lot and spot management, vehicle data, reservations, sensor occupancy detection, and automated enforcement into a unified platform. Without such integration, parking operations lack efficiency, transparency, and scalability.

The proposed UMBC Parking Management System addresses this issue by implementing a PostgreSQL-backed database that manages parking as a real-time, data-driven ecosystem.

## 1.2 Parking Users

The system supports multiple categories of users:

- **Commuter Students** – Purchase student permits and park in designated student lots.
- **Faculty and Staff** – Hold faculty/staff permits and access restricted premium zones.
- **Visitors and Guest Lecturers** – Reserve temporary parking spaces online and pay per visit.

## 1.3 Scope of Implementation

This project will implement a PostgreSQL database system that includes:

1. **User and Vehicle Management**
    - Store user profiles (students, faculty, visitors).
    - Associate multiple vehicles with individual users.
    - Store license plate information for validation.
2. **Permit Management**
    - Define permit types (student, faculty, visitor).

- Assign permits to users.
- Enforce expiration dates.
- Restrict lot and zone access by permit type.

3. **Lot and Spot Management**
   - Define parking lots, zones, rows, and individual spots.
   - Track real-time occupancy per spot.
   - Associate spots with allowed permit types.
4. **Reservation System**
   - Allow visitors to reserve specific spots.
   - Prevent double-booking through constraints and transactions.
   - Lock spots upon confirmed payment.
5. **Sensor Integration**
   - Update occupancy when a ground sensor detects a vehicle.
   - Validate vehicle authorization against zone restrictions.
6. **Automated Ticketing**
   - Generate violations for unauthorized parking.
   - Store fine amounts and violation details.
   - Record notification events.
7. **Payments**
   - Record ticket payments.
   - Track outstanding balances.
8. **Reporting and Analytics**
   - Lot utilization rates.
   - Revenue reports.
   - Violation frequency reports.
   - Active and expired permit tracking.
   - Peak usage analysis.

9. **Concurrency Control**
    - Prevent race conditions during reservation.

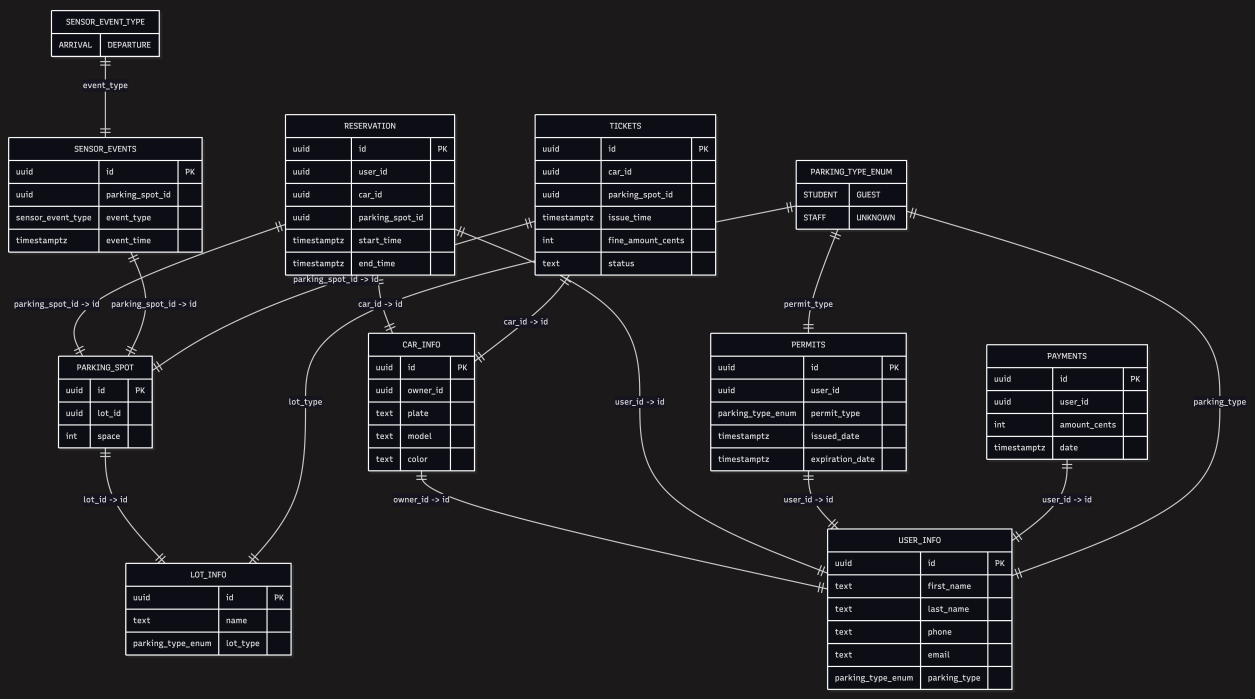
This scope focuses specifically on the database design, implementation, constraints, indexing, realistic operations, and concurrency control. It does not include a full mobile or web application interface.

## 1.4 Concrete Use Cases

The database must support the following operations:

- Register a new user (student, faculty, or visitor).
- Add one or more vehicles to a user account.
- Purchase and assign a parking permit to a user.
- Define parking lots and individual parking spots.
- Associate permit types with allowed zones.
- Query real-time available parking spots in a specific lot.
- Create a visitor reservation for a specific date and time.
- Prevent double-booking of a parking spot.
- Update occupancy when a sensor detects a vehicle.
- Validate a vehicle's permit against the spot's allowed zone.
- Automatically generate a ticket for unauthorized parking.
- Record ticket payments and mark violations as resolved.
- Generate reports on lot occupancy over time.
- Generate revenue reports from permits, reservations, and fines.
- Detect expired permits and flag vehicles as invalid.
- Handle simultaneous reservation attempts without data corruption.

# HW #2 Requirements & Conceptual Design

## 2.1 ER Diagram

To see full image click on `parking-er-diagram.png`